LSTM Weekly Temperature Model - Kaggle Training Notebook

Notebook nay ghi toan bo source code vao `/kaggle/working/lstm_temp_weekly/` roi chay `main.py` de huan luyen mo hinh LSTM **Direct Multi-step** du bao nhiet do (`temperature_celsius`) theo gio cho ca **168 gio (1 tuan)** tiep theo tai cac diem luoi nam trong lanh tho dat lien Viet Nam, bao gom diem sat bien gioi, sat bo bien va cac diem noi dia.

Yeu cau: them dataset Parquet tai `/kaggle/input/datasets/nguyentrangggggg/vietnam-meteorological-weather-data-parquet/weather.parquet`, them GeoPackage GADM level-0 tai `/kaggle/input/datasets/nglan271204/vnm-gpkg/gadm41_VNM.gpkg` va bat **GPU T4** trong Settings.

In [ ]:
```
!pip install -q seaborn geopandas shapely pyogrio
```

In [ ]:
```
import os

os.makedirs('/kaggle/working/data', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/data', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/preprocessing', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/dataset', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/model', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/training', exist_ok=True)
os.makedirs('/kaggle/working/lstm_temp_weekly/visualization', exist_ok=True)

print('All directories created.')
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/data/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/data/load_data.py
"""
Nap du lieu thoi tiet cho cac diem luoi nam trong lanh tho Viet Nam.

Luon loc toa do truoc khi doc day du du lieu:
  1. Chi doc hai cot latitude/longitude de lay danh sach toa do duy nhat.
  2. Dung GeoPackage GADM level-0 de giu cac toa do nam trong polygon Viet Nam
     da buffer 0.05 do.
  3. Moi doc day du _COLS tu Parquet voi filters theo cac toa do hop le.
  4. Sap xep chuoi thoi gian va ep float64 -> float32.
"""
from pathlib import Path
from typing import Any, cast

import geopandas as gpd
```

```python
import pandas as pd
from shapely.geometry import Point

DATA_PATH = '/kaggle/input/datasets/nguyentrangggggg/vietnam-meteorological-weather-
GPKG_PATH = '/kaggle/input/datasets/nglan271204/vnm-gpkg/gadm41_VNM.gpkg'
LOCAL_GPKG_PATH = './data/gadm41_VNM.gpkg'
LAND_BUFFER_DEGREES = 0.05

target_cols = ['temperature_celsius']

_COORD_COLS = ['latitude', 'longitude']

_COLS = [
    'latitude', 'longitude', 'valid_time',
    'temperature_celsius', 'apparent_temperature',
    'relative_humidity', 'wind_speed', 'wind_direction',
    'total_precipitation', 'total_cloud_cover',
    'mean_sea_level_pressure', 'surface_pressure',
    'sea_surface_temperature', 'air_density',
]


def _resolve_gpkg_path(gpkg_path: str) -> str:
    """Uu tien duong dan Kaggle, fallback ve ./data khi test local."""
    if Path(gpkg_path).exists():
        return gpkg_path
    if Path(LOCAL_GPKG_PATH).exists():
        return LOCAL_GPKG_PATH
    return gpkg_path


def _union_geometries(geometry):
    """Gop tat ca polygon Viet Nam thanh mot hinh hoc duy nhat."""
    return geometry.union_all()


def _load_vietnam_land_geometry(gpkg_path: str = GPKG_PATH):
    """Doc polygon quoc gia Viet Nam tu GeoPackage GADM level-0."""
    gpkg_path = _resolve_gpkg_path(gpkg_path)
    print(f"Dang doc ranh gioi Viet Nam tu GeoPackage: {gpkg_path}")
    vietnam_level0 = gpd.read_file(gpkg_path, layer=0)

    if vietnam_level0.empty:
        raise ValueError("GeoPackage khong co hinh hoc Viet Nam o layer=0.")

    # Du lieu thoi tiet la kinh/vi do, nen dua polygon ve EPSG:4326 neu can.
    if vietnam_level0.crs is not None and vietnam_level0.crs.to_epsg() != 4326:
        vietnam_level0 = vietnam_level0.to_crs(epsg=4326)

    vietnam_geom = _union_geometries(vietnam_level0.geometry)

    if vietnam_geom.is_empty:
        raise ValueError("Khong tao duoc polygon Viet Nam tu GeoPackage.")

    return vietnam_geom
```

```python
def _read_unique_coordinates(data_path: str) -> pd.DataFrame:
    """
    Chi doc hai cot toa do tu Parquet.

    Buoc nay khong doc cac cot khi tuong lon nhu nhiet do/mua/ap suat, nen RAM
    nhe hon rat nhieu so voi doc full _COLS truoc roi moi loc.
    """
    print("Dang doc nhe 2 cot latitude/longitude de lay toa do duy nhat ...")
    coords = pd.read_parquet(data_path, columns=_COORD_COLS, engine='auto')
    unique_coords = coords.drop_duplicates().reset_index(drop=True)
    print(f"So diem toa do duy nhat truoc khi loc: {len(unique_coords):,}")
    return unique_coords


def _find_vietnam_land_coordinates(
    unique_coords: pd.DataFrame,
    vietnam_geom,
    buffer_degrees: float = LAND_BUFFER_DEGREES,
) -> pd.DataFrame:
    """
    Loc toa do nam trong polygon Viet Nam da buffer.

    Chi tao Point va contains() cho danh sach toa do duy nhat, tuyet doi khong
    chay phep toan hinh hoc tren hang trieu dong du lieu thoi gian.
    """
    vietnam_area = vietnam_geom.buffer(buffer_degrees)

    # Shapely Point dung thu tu (x, y) = (longitude, latitude).
    points = [
        Point(lon, lat)
        for lat, lon in unique_coords[_COORD_COLS].itertuples(index=False, name=Non
    ]
    inside_mask = [vietnam_area.contains(point) for point in points]

    land_coords = unique_coords.loc[inside_mask, _COORD_COLS].copy()
    print(
        "So diem toa do nam trong lanh tho Viet Nam sau khi loc "
        f"(buffer={buffer_degrees} do): {len(land_coords):,}"
    )

    if land_coords.empty:
        raise ValueError(
            "Khong co diem luoi nao nam trong polygon Viet Nam. "
            "Hay kiem tra CRS/toa do hoac duong dan GeoPackage."
        )

    return land_coords


def _build_coordinate_filters(keep_coords: pd.DataFrame) -> list:
    """
    Tao Parquet filters theo cap toa do hop le.

    Dung OR theo tung latitude, trong moi latitude dung longitude in [...]
    de filter dung cap (latitude, longitude) nhung ngan gon hon hang nghin
```

```
    dieu kien OR rieng le.
    """
    coords = keep_coords[_COORD_COLS].astype('float64')
    lon_by_lat: dict[float, list[float]] = {}

    for lat_value, lon_value in coords.itertuples(index=False, name=None):
        lat = float(cast(Any, lat_value))
        lon = float(cast(Any, lon_value))
        lon_by_lat.setdefault(lat, []).append(lon)

    return [
        [
            ('latitude', '==', lat),
            ('longitude', 'in', lons),
        ]
        for lat, lons in lon_by_lat.items()
    ]


def _read_filtered_weather_data(data_path: str, keep_coords: pd.DataFrame) -> pd.Da
    """Doc day du _COLS chi cho cac toa do dat lien da duoc loc truoc."""
    filters = _build_coordinate_filters(keep_coords)
    print(f"Dang doc Parquet voi filters cho {len(keep_coords):,} diem toa do hop l
    return pd.read_parquet(
        data_path,
        columns=_COLS,
        engine='auto',
        filters=filters,
    )


def load_data(
    data_path: str = DATA_PATH,
    gpkg_path: str = GPKG_PATH,
    buffer_degrees: float = LAND_BUFFER_DEGREES,
) -> pd.DataFrame:
    vietnam_geom = _load_vietnam_land_geometry(gpkg_path)
    unique_coords = _read_unique_coordinates(data_path)
    land_coords = _find_vietnam_land_coordinates(unique_coords, vietnam_geom, buffe
    df = _read_filtered_weather_data(data_path, land_coords)

    # valid_time phai la datetime de tao feature thoi gian dung ve sau.
    df['valid_time'] = pd.to_datetime(df['valid_time'])

    # Sap xep de moi cap toa do co chuoi thoi gian lien tuc truoc khi tao sequence.
    df = df.sort_values(['latitude', 'longitude', 'valid_time']).reset_index(drop=T

    # float64 -> float32 giup giam RAM khi feature engineering va train LSTM.
    for col in df.select_dtypes('float64').columns:
        df[col] = df[col].astype('float32')

    n_locs = df.groupby(['latitude', 'longitude'], sort=False).ngroups
    print(
        "Nap du lieu dat lien Viet Nam thanh cong! "
        f"{len(df):,} dong | {n_locs:,} diem toa do | target_cols={target_cols}"
```

```
    )
    return df
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/preprocessing/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/preprocessing/feature_engineering.py
"""
Task: Build the feature matrix for weekly TEMPERATURE forecasting.

Pipeline order (each step depends on the previous one):
  1. Interpolate raw meteorological NaN (per coordinate group).
  2. NO log1p — temperature (Celsius) is already a near-symmetric, bell-shaped
     quantity, so it is kept in its native physical unit.
  3. YEARLY feature   : 'year_normalized' = (year - 2020) / 6  → long-term climate
  4. MONTHLY feature  : 'monthly_mean_temperature' = historical mean temperature pe
                        (lat, lon, month) → a climatology baseline telling the mod
                        which months are winter / summer for each region. This is
                        extremely strong prior for the seasonal cycle.
  5. WEEKLY context   : lag (1, 3, 6, 24, 168h) and rolling-mean (3, 6, 24, 168h)
                        features → the model "remembers" the past week.
  6. Cyclic time encodings (hour, day_of_week, month, day_of_year as sin/cos).
  7. Drop residual NaN rows created by the lag/rolling boundaries.

The target lives in raw Celsius the whole way through, so every temperature-derived
feature and baseline shares one consistent physical space.
"""
import numpy as np
import pandas as pd

TARGET_COL = 'temperature_celsius'

_METEO_COLS = [
    'temperature_celsius', 'apparent_temperature',
    'relative_humidity', 'wind_speed', 'wind_direction',
    'total_precipitation', 'total_cloud_cover',
    'mean_sea_level_pressure', 'surface_pressure',
    'sea_surface_temperature', 'air_density',
]

_LAG_HOURS  = [1, 3, 6, 24, 168]
_ROLL_HOURS = [3, 6, 24, 168]


# ── 1. Interpolate raw NaN ──────────────────────────────────────────────

def _interpolate_raw(df: pd.DataFrame) -> pd.DataFrame:
    existing = [c for c in _METEO_COLS if c in df.columns]
    df[existing] = (
        df.groupby(['latitude', 'longitude'], sort=False)[existing]
        .transform(lambda x: x.interpolate(method='linear', limit_direction='both')
    )
    return df
```

```python
# ── 2. Yearly climate-drift feature ──────────────────────────────────────

def _add_year_feature(df: pd.DataFrame) -> pd.DataFrame:
    df['year_normalized'] = ((df['valid_time'].dt.year - 2020) / 6).astype('float32
    return df


# ── 3. Monthly climatology baseline ──────────────────────────────────────

def _add_monthly_climatology(df: pd.DataFrame) -> pd.DataFrame:
    month = df['valid_time'].dt.month
    df['monthly_mean_temperature'] = (
        df.groupby(['latitude', 'longitude', month], sort=False)[TARGET_COL]
        .transform('mean')
        .astype('float32')
    )
    return df


# ── 4. Weekly context: lags + rolling means ──────────────────────────────

def _add_lag_features(df: pd.DataFrame) -> pd.DataFrame:
    grp = df.groupby(['latitude', 'longitude'], sort=False)[TARGET_COL]
    for h in _LAG_HOURS:
        df[f'{TARGET_COL}_lag{h}'] = grp.shift(h).astype('float32')
    return df


def _add_rolling_features(df: pd.DataFrame) -> pd.DataFrame:
    grp = df.groupby(['latitude', 'longitude'], sort=False)[TARGET_COL]
    for h in _ROLL_HOURS:
        # shift(1) guarantees the current hour never leaks into its own feature.
        df[f'{TARGET_COL}_roll{h}'] = (
            grp.transform(lambda x: x.shift(1).rolling(h, min_periods=1).mean())
            .astype('float32')
        )
    return df


# ── 5. Cyclic time encodings ─────────────────────────────────────────────

def _add_time_features(df: pd.DataFrame) -> pd.DataFrame:
    dt = df['valid_time']
    df['hour_sin']        = np.sin(2 * np.pi * dt.dt.hour      / 24 ).astype('float
    df['hour_cos']        = np.cos(2 * np.pi * dt.dt.hour      / 24 ).astype('float
    df['day_of_week_sin'] = np.sin(2 * np.pi * dt.dt.dayofweek / 7  ).astype('float
    df['day_of_week_cos'] = np.cos(2 * np.pi * dt.dt.dayofweek / 7  ).astype('float
    df['month_sin']       = np.sin(2 * np.pi * dt.dt.month     / 12 ).astype('float
    df['month_cos']       = np.cos(2 * np.pi * dt.dt.month     / 12 ).astype('float
    df['day_of_year_sin'] = np.sin(2 * np.pi * dt.dt.dayofyear / 365).astype('float
    df['day_of_year_cos'] = np.cos(2 * np.pi * dt.dt.dayofyear / 365).astype('float
    return df


# ── Orchestrator ─────────────────────────────────────────────────────────
```

```python
def engineer_features(df: pd.DataFrame) -> pd.DataFrame:
    df = _interpolate_raw(df)
    df = _add_year_feature(df)
    df = _add_monthly_climatology(df)
    df = _add_lag_features(df)
    df = _add_rolling_features(df)
    df = _add_time_features(df)
    df = df.dropna().reset_index(drop=True)
    return df
```

In [ ]:
```python
%%writefile /kaggle/working/lstm_temp_weekly/preprocessing/scaling.py
"""
Task: Fit a MinMaxScaler on all numeric feature columns, scale the DataFrame
      in-place to [0, 1], and persist both the scaler and the feature-column
      list to disk so inference can reproduce the exact transform.
"""
import pickle
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler

SCALER_PATH       = 'scaler_temp.pkl'
FEATURE_COLS_PATH = 'feature_cols_temp.pkl'

# Coordinates / time are identifiers, not model inputs → never scaled.
_EXCLUDE = {'latitude', 'longitude', 'valid_time'}


def fit_and_scale(df: pd.DataFrame):
    feature_cols = [c for c in df.columns if c not in _EXCLUDE]

    scaler = MinMaxScaler(feature_range=(0, 1))
    scaled = scaler.fit_transform(df[feature_cols].values.astype(np.float32))
    df[feature_cols] = scaled.astype(np.float32)

    with open(SCALER_PATH, 'wb') as f:
        pickle.dump(scaler, f)
    with open(FEATURE_COLS_PATH, 'wb') as f:
        pickle.dump(feature_cols, f)

    print(f"  Scaler          -> {SCALER_PATH}")
    print(f"  Feature columns -> {FEATURE_COLS_PATH}  ({len(feature_cols)} cols)")

    return df, scaler, feature_cols
```

In [ ]:
```python
%%writefile /kaggle/working/lstm_temp_weekly/dataset/__init__.py
# package marker
```

In [ ]:
```python
%%writefile /kaggle/working/lstm_temp_weekly/dataset/sequence_dataset.py
"""
Task: Turn the scaled DataFrame into per-coordinate numpy arrays and build
      lazy-loading sequence indices, split temporally per coordinate
      (80% train / 20% test) so there is zero geographic leakage.

Forecasting setup (Seq2Seq Encoder-Decoder):
```

```
  x      : 168 past hours  × N features           → encoder input,  shape (168, N)
  y_feat : 168 future hours × decoder features     → decoder exog.,  shape (168, F
  y      : 168 future hours of temperature         → target,         shape (168,)

  where for a window starting at `start`:
      x      = arr[start            : start+seq_len]
      future = arr[start+seq_len    : start+seq_len+predict_steps]
      y_feat = future[:, decoder_feat_idx]   (known-in-advance calendar/climatology
      y      = future[:, target_idx]

`y_feat` carries only the deterministic, future-known features (cyclic time
encodings + monthly climatology + year drift). The decoder consumes these
alongside the previous-step temperature, which is what lets the model
reconstruct the diurnal (day/night) peaks instead of smoothing them away.

Memory strategy: coordinate arrays are stored once; every sample is just a
(coord_idx, start) pointer. Sequences are sliced lazily in __getitem__, so we
never materialise the full 3-D tensor (which would blow up RAM for all of VN).
"""
import numpy as np
import torch
from torch.utils.data import Dataset

SEQUENCE_LENGTH = 168   # look back 7 days (168 hours)
PREDICT_STEPS   = 168   # forecast the next 7 days (168 hours)

# Features the model legitimately KNOWS for the future window (calendar is
# deterministic; monthly climatology / year drift are per-(coord, month) priors).
# These — and only these — are fed to the decoder at every future step.
DECODER_FEATURE_COLS = [
    'hour_sin', 'hour_cos',
    'day_of_week_sin', 'day_of_week_cos',
    'month_sin', 'month_cos',
    'day_of_year_sin', 'day_of_year_cos',
    'year_normalized', 'monthly_mean_temperature',
]


# ── Coordinate array builder ──────────────────────────────────────────────

def build_coordinate_arrays(df, feature_cols: list):
    """
    Return:
      arrays : list of float32 arrays, one per (lat, lon), sorted by coordinate.
      coords : parallel list of (lat, lon) tuples (used by the spatial error map).
    """
    arrays, coords = [], []
    for (lat, lon), grp in df.groupby(['latitude', 'longitude'], sort=True):
        arrays.append(grp[feature_cols].values.astype(np.float32))
        coords.append((float(lat), float(lon)))
    return arrays, coords


# ── Decoder feature index helper ──────────────────────────────────────────

def resolve_decoder_feat_idx(feature_cols: list) -> list:
```

```
    """Map DECODER_FEATURE_COLS → their column positions inside feature_cols."""
    return [feature_cols.index(c) for c in DECODER_FEATURE_COLS if c in feature_col


# ── Temporal split (per coordinate, no cross-boundary contamination) ──────────

def build_split_indices(
    coordinate_arrays: list,
    seq_len: int       = SEQUENCE_LENGTH,
    predict_steps: int = PREDICT_STEPS,
    train_ratio: float = 0.8,
):
    """
    Split each coordinate's time axis at 80%.
      Train samples : the whole window [start, start+seq_len+predict_steps) < split
      Test  samples : windows that start at/after split → no leakage from training.
    A sample needs seq_len + predict_steps consecutive hours to be valid.
    """
    span = seq_len + predict_steps
    train_idx, test_idx = [], []

    for cid, arr in enumerate(coordinate_arrays):
        n     = len(arr)
        split = int(n * train_ratio)

        # train: last index touched = start + span - 1 < split
        for start in range(0, max(0, split - span + 1)):
            train_idx.append((cid, start))

        # test: window starts at/after split, still fits inside n
        for start in range(split, max(split, n - span + 1)):
            test_idx.append((cid, start))

    return train_idx, test_idx


# ── Dataset ──────────────────────────────────────────────────────────────────

class WeatherSequenceDataset(Dataset):
    """
    Lazy multi-step sequence dataset for the Seq2Seq model. Each __getitem__
    slices one (x, y_feat, y) triple on the fly — no pre-materialised 3-D tensor.

        x      : (seq_len, n_features)        encoder input
        y_feat : (predict_steps, n_dec_feat)  known future decoder features
        y      : (predict_steps,)             future temperature (MinMax scaled °C)
    """

    def __init__(
        self,
        coordinate_arrays: list,
        indices: list,
        target_idx: int,
        decoder_feat_idx: list,
        seq_len: int       = SEQUENCE_LENGTH,
        predict_steps: int = PREDICT_STEPS,
```

```
    ):
        self.coordinate_arrays = coordinate_arrays
        self.indices           = indices
        self.target_idx        = target_idx
        self.decoder_feat_idx  = np.asarray(decoder_feat_idx, dtype=np.int64)
        self.seq_len           = seq_len
        self.predict_steps     = predict_steps

    def __len__(self) -> int:
        return len(self.indices)

    def __getitem__(self, idx):
        cid, start = self.indices[idx]
        arr = self.coordinate_arrays[cid]

        x_end = start + self.seq_len
        y_end = x_end + self.predict_steps

        # .copy() → independent buffers so DataLoader workers stay safe.
        x      = arr[start:x_end].copy()                        # (seq_len, N)
        future = arr[x_end:y_end]                               # (predict_steps, N)
        y_feat = future[:, self.decoder_feat_idx].copy()        # (predict_steps, F)
        y      = future[:, self.target_idx].copy()              # (predict_steps,)

        return (
            torch.from_numpy(x),
            torch.from_numpy(y_feat),
            torch.from_numpy(y),
        )
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/model/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/model/lstm_model.py
"""
Task: Define the multi-step LSTM model.

Architecture (Seq2Seq Encoder-Decoder — LSTM only, no GRU):

  Encoder : nn.LSTM over the 168 past hours → final (h, c) = context vector.
  Decoder : a SEPARATE nn.LSTM initialised with the encoder's (h, c). At every
            future step its input is
                [ temperature of the previous step ]  ⊕  [ known future features ]
            and a Linear(hidden, 1) head emits the next temperature.

Two forward modes share one set of weights:

  • Vectorized Teacher Forcing (training, ratio = 1.0):
      The whole ground-truth target, shifted one step and prefixed with the last
      observed temperature, is fed to the decoder LSTM in a SINGLE call. PyTorch
      unrolls the recurrence on the GPU in parallel → no Python for-loop, fast.

  • Autoregressive (evaluation / inference):
      The decoder consumes its OWN previous prediction at each step, looping over
      the 168 horizon — the honest deployment-time behaviour.
```

```
Feeding the known future calendar features to the decoder is what preserves the
day/night diurnal peaks that the old Direct (flat Linear) head smoothed away.
"""
import torch
import torch.nn as nn


class LSTMModel(nn.Module):
    def __init__(
        self,
        input_size:    int,
        dec_feat_size: int,
        hidden_size:   int   = 128,
        num_layers:    int   = 2,
        predict_steps: int   = 168,
        dropout:       float = 0.1,
        target_idx:    int   = 0,
    ):
        super().__init__()
        self.predict_steps = predict_steps
        self.target_idx    = target_idx

        # PyTorch applies dropout between stacked LSTM layers only.
        layer_dropout = dropout if num_layers > 1 else 0.0

        self.encoder = nn.LSTM(
            input_size  = input_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            dropout     = layer_dropout,
            batch_first = True,
        )
        # decoder input per step = [prev temperature (1)] ⊕ [known future features
        self.decoder = nn.LSTM(
            input_size  = 1 + dec_feat_size,
            hidden_size = hidden_size,
            num_layers  = num_layers,
            dropout     = layer_dropout,
            batch_first = True,
        )
        self.out = nn.Linear(hidden_size, 1)

    def forward(self, x, y_feat, y=None, teacher_forcing=True):
        # x:      (batch, seq_len, input_size)
        # y_feat: (batch, predict_steps, dec_feat_size)
        # y:      (batch, predict_steps)            ground-truth target (TF only)
        _, (h, c) = self.encoder(x)                          # context vector
        last_temp = x[:, -1, self.target_idx].unsqueeze(1)   # (B, 1) start token

        if teacher_forcing:
            # ── Vectorized Teacher Forcing: one parallel decoder call ──
            prev   = torch.cat([last_temp, y[:, :-1]], dim=1)         # (B, T) shif
            dec_in = torch.cat([prev.unsqueeze(-1), y_feat], dim=-1)  # (B, T, 1+F)
            dec_out, _ = self.decoder(dec_in, (h, c))                 # (B, T, hida
            return self.out(dec_out).squeeze(-1)                      # (B, T)
```

```
        # ── Autoregressive: feed own prediction back, step by step ──
        prev  = last_temp                                    # (B, 1)
        preds = []
        for t in range(self.predict_steps):
            step_in = torch.cat([prev, y_feat[:, t]], dim=-1).unsqueeze(1)  # (B,1,
            out_t, (h, c) = self.decoder(step_in, (h, c))
            prev = self.out(out_t).squeeze(1)                # (B, 1)
            preds.append(prev)
        return torch.cat(preds, dim=1)                       # (B, T)
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/training/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/training/trainer.py
"""
Task: Run the Seq2Seq training loop — one epoch of gradient updates over the
      train set, then inference over the val set, logging MSE / RMSE / MAE.

Speed & regularisation:
  • Mixed Precision (torch.amp) shrinks T4 step time and memory.
  • Early Stopping (patience) halts as soon as val_loss stops improving — the old
    Direct model overfit by epoch 1-2, so this saves hours.
  • A checkpoint is written every epoch to models/<prefix>_epoch_NN.pt (full
    state_dict + optimizer + network config + epoch + metrics), and the BEST
    epoch's weights are restored into the model before returning.

Both train and val passes use TEACHER FORCING here (a single parallel decoder
call) so the per-epoch monitor is fast. The honest autoregressive °C error is
reported separately by the evaluator. Metrics here live in the model's working
space (MinMax-scaled [0, 1]), matching the optimisation objective.
"""
import copy
import os

import numpy as np
import torch


# ── Single epoch pass ──────────────────────────────────────────────────────

def _run_epoch(model, loader, optimizer, criterion, device, train, scaler=None):
    model.train(train)
    use_amp = scaler is not None and scaler.is_enabled()
    sse = sae = count = 0.0   # streaming accumulators → no giant pred buffers

    with torch.set_grad_enabled(train):
        for X_b, yf_b, y_b in loader:
            X_b  = X_b.to(device, non_blocking=True)
            yf_b = yf_b.to(device, non_blocking=True)
            y_b  = y_b.to(device, non_blocking=True)

            if train:
                optimizer.zero_grad(set_to_none=True)
                with torch.amp.autocast(device.type, enabled=use_amp):
```

```
                    out  = model(X_b, yf_b, y_b, teacher_forcing=True)   # (B, 168)
                    loss = criterion(out, y_b)
                scaler.scale(loss).backward()
                scaler.step(optimizer)
                scaler.update()
            else:
                with torch.amp.autocast(device.type, enabled=use_amp):
                    out = model(X_b, yf_b, y_b, teacher_forcing=True)

            diff   = (out.float() - y_b).detach()
            sse   += torch.sum(diff ** 2).item()
            sae   += torch.sum(torch.abs(diff)).item()
            count += diff.numel()

    mse  = sse / count
    rmse = float(np.sqrt(mse))
    mae  = sae / count
    return mse, rmse, mae


# ── Full training loop ──────────────────────────────────────────────

def train_model(model, train_loader, val_loader, optimizer, criterion, device,
                epochs: int = 5, patience: int = 2, config: dict = None,
                models_dir: str = 'models', ckpt_prefix: str = 'lstm_temp_weekly',
                use_amp: bool = True) -> dict:
    history = {k: [] for k in [
        'epoch',
        'train_mse', 'train_rmse', 'train_mae',
        'val_mse',   'val_rmse',   'val_mae',
    ]}

    os.makedirs(models_dir, exist_ok=True)
    amp_on = use_amp and device.type == 'cuda'
    scaler = torch.amp.GradScaler(device.type, enabled=amp_on)
    if amp_on:
        print("  Mixed precision (AMP) : ENABLED")

    best_val   = float('inf')
    best_epoch = 0
    best_state = copy.deepcopy(model.state_dict())
    no_improve = 0

    for epoch in range(1, epochs + 1):
        tr_mse, tr_rmse, tr_mae = _run_epoch(
            model, train_loader, optimizer, criterion, device, True, scaler
        )
        vl_mse, vl_rmse, vl_mae = _run_epoch(
            model, val_loader, None, criterion, device, False, scaler
        )

        for key, val in zip(history.keys(), [
            epoch,
            tr_mse, tr_rmse, tr_mae,
            vl_mse, vl_rmse, vl_mae,
        ]):
```

```
            history[key].append(val)

        # ── Per-epoch checkpoint ──────────────────────────────────────────
        ckpt_path = os.path.join(models_dir, f"{ckpt_prefix}_epoch_{epoch:02d}.pt")
        torch.save(
            {
                'epoch':                epoch,
                'model_state_dict':     model.state_dict(),
                'optimizer_state_dict': optimizer.state_dict(),
                'config':               config or {},
                'metrics': {
                    'train_mse': tr_mse, 'train_rmse': tr_rmse, 'train_mae': tr_mae
                    'val_mse':   vl_mse, 'val_rmse':   vl_rmse, 'val_mae':   vl_mae
                },
            },
            ckpt_path,
        )

        improved = vl_mse < best_val
        if improved:
            best_val, best_epoch = vl_mse, epoch
            best_state = copy.deepcopy(model.state_dict())
            no_improve = 0
        else:
            no_improve += 1

        print(
            f"Epoch [{epoch:02d}/{epochs}]  "
            f"Train MSE: {tr_mse:.5f}, Val MSE: {vl_mse:.5f}  |  "
            f"Val RMSE: {vl_rmse:.5f}, Val MAE: {vl_mae:.5f}  "
            f"{'  <-- best' if improved else f'  (no improve {no_improve}/{patience
            f"  -> {ckpt_path}"
        )

        # ── Early stopping ────────────────────────────────────────────────
        if no_improve >= patience:
            print(f"  Early stopping at epoch {epoch} "
                  f"(best epoch {best_epoch}, val MSE {best_val:.5f}).")
            break

    # Restore best weights so downstream evaluation uses the best epoch.
    model.load_state_dict(best_state)
    print(f"  Restored best weights from epoch {best_epoch} (val MSE {best_val:.5f}
    history['best_epoch'] = best_epoch
    return history
```

In [ ]:

```
%%writefile /kaggle/working/lstm_temp_weekly/training/evaluator.py
"""
Task: Evaluate the trained Seq2Seq model on the held-out test set and report
      errors in REAL temperature units (°C).

Inference is AUTOREGRESSIVE (teacher_forcing=False): the decoder feeds its own
previous prediction back at each of the 168 future steps — the honest
deployment-time behaviour, unlike the teacher-forced monitor used in training.

The target was trained as MinMax-scaled temperature in [0, 1]. Because there is
```

```
NO log transform for temperature, inversion is a single step:
    °C = scaled * data_range_ + data_min_     (for the target column only)
Both predictions and ground truth are inverted before computing RMSE / MAE.
"""
import numpy as np
import torch
from torch.utils.data import DataLoader

from dataset.sequence_dataset import WeatherSequenceDataset, DECODER_FEATURE_COLS


# ── Target inverse transform (scaled → °C) ──────────────────────────────

def inverse_target(scaled, scaler, target_idx: int) -> np.ndarray:
    """Invert MinMax scaling for the temperature target column (no log step)."""
    scaled = np.asarray(scaled, dtype=np.float64)
    return scaled * scaler.data_range_[target_idx] + scaler.data_min_[target_idx]


def _inverse_col(scaled, scaler, col_idx: int) -> np.ndarray:
    """Invert MinMax scaling for an arbitrary feature column."""
    scaled = np.asarray(scaled, dtype=np.float64)
    return scaled * scaler.data_range_[col_idx] + scaler.data_min_[col_idx]


# ── Test-set metrics (streaming, in °C) ─────────────────────────────────

def evaluate_test(model, test_dataset, device, scaler, target_idx: int,
                  batch_size: int = 2048) -> tuple:
    loader = DataLoader(
        test_dataset, batch_size=batch_size, shuffle=False,
        num_workers=2, pin_memory=(device.type == 'cuda'),
    )

    model.eval()
    sse = sae = count = 0.0

    with torch.no_grad():
        for X_b, yf_b, y_b in loader:
            X_b  = X_b.to(device, non_blocking=True)
            yf_b = yf_b.to(device, non_blocking=True)
            out  = model(X_b, yf_b, teacher_forcing=False).cpu().numpy()

            preds_c   = inverse_target(out,         scaler, target_idx)
            targets_c = inverse_target(y_b.numpy(), scaler, target_idx)

            diff   = preds_c - targets_c
            sse   += float(np.sum(diff ** 2))
            sae   += float(np.sum(np.abs(diff)))
            count += diff.size

    mse  = sse / count
    rmse = float(np.sqrt(mse))
    mae  = sae / count

    w = 56
```

```
    print()
    print("=" * w)
    print(f"{'TEST SET EVALUATION  (real units, °C)':^{w}}")
    print("=" * w)
    print(f"  {'Metric':<26} {'Value':>16}")
    print("-" * w)
    print(f"  {'Test MSE  (°C^2)':<26} {mse:>16.6f}")
    print(f"  {'Test RMSE (°C)':<26} {rmse:>16.6f}")
    print(f"  {'Test MAE  (°C)':<26} {mae:>16.6f}")
    print("=" * w)

    return mse, rmse, mae


# ── Prediction collector for visualisation (subsampled, in °C) ──────────────

def collect_predictions(model, coordinate_arrays, indices, target_idx, decoder_feat
                        feature_cols, scaler, seq_len, predict_steps, device,
                        max_samples: int = 4000, batch_size: int = 2048):
    """
    Run AUTOREGRESSIVE inference over a random subset of `indices` and return
    real-unit arrays for plotting:
        preds_c       : (M, predict_steps)   °C
        targets_c     : (M, predict_steps)   °C
        coord_ids     : (M,)                 coordinate index of each sample
        hours_of_day  : (M, predict_steps)   clock hour 0..23 of each forecast step
    Subsampled to keep the plotting buffers small while covering many regions.

    `hours_of_day` is recovered from the (scaled) hour_sin/hour_cos columns in
    y_feat — inverse-scaled, then hour = atan2(sin, cos) / 2π * 24.
    """
    rng = np.random.default_rng(42)
    if len(indices) > max_samples:
        pick = rng.choice(len(indices), size=max_samples, replace=False)
        pick.sort()
        sub_indices = [indices[i] for i in pick]
    else:
        sub_indices = list(indices)

    ds = WeatherSequenceDataset(coordinate_arrays, sub_indices, target_idx,
                                decoder_feat_idx, seq_len, predict_steps)
    loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
                        num_workers=2, pin_memory=(device.type == 'cuda'))

    # Locate hour_sin/hour_cos: position inside y_feat AND inside feature_cols.
    hs_feat = feature_cols.index('hour_sin')
    hc_feat = feature_cols.index('hour_cos')
    hs_col  = list(decoder_feat_idx).index(hs_feat)
    hc_col  = list(decoder_feat_idx).index(hc_feat)

    model.eval()
    preds_buf, tgts_buf, hour_buf = [], [], []
    with torch.no_grad():
        for X_b, yf_b, y_b in loader:
            X_b  = X_b.to(device, non_blocking=True)
            yf_d = yf_b.to(device, non_blocking=True)
```

```
            preds_buf.append(model(X_b, yf_d, teacher_forcing=False).cpu().numpy())
            tgts_buf.append(y_b.numpy())

            yf_np = yf_b.numpy()
            sin   = _inverse_col(yf_np[:, :, hs_col], scaler, hs_feat)
            cos   = _inverse_col(yf_np[:, :, hc_col], scaler, hc_feat)
            hours = (np.round(np.arctan2(sin, cos) / (2 * np.pi) * 24).astype(int))
            hour_buf.append(hours)

    preds_c      = inverse_target(np.concatenate(preds_buf), scaler, target_idx)
    targets_c    = inverse_target(np.concatenate(tgts_buf),  scaler, target_idx)
    coord_ids    = np.array([cid for cid, _ in sub_indices], dtype=np.int64)
    hours_of_day = np.concatenate(hour_buf)

    return preds_c, targets_c, coord_ids, hours_of_day
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/visualization/__init__.py
# package marker
```

In [ ]:
```
%%writefile /kaggle/working/lstm_temp_weekly/visualization/plot_loss.py
"""
Task: Persist the training history CSV and render the report-grade figures into
      the 'plots/' directory.

All temperature quantities passed in here are already in REAL units (°C) — the
caller inverts MinMax via training.evaluator.inverse_target first.

Figures
  1-3. plots/loss_mse.png, loss_rmse.png, loss_mae.png  — Train vs Val curves.
  4.   plots/scatter_actual_vs_pred.png                 — actual vs predicted (+ R²
  5.   plots/sample_prediction_comparison.png           — South vs North station, 1
  6.   plots/residuals_histogram.png                    — distribution of (actual -
  7.   plots/feature_correlation_heatmap.png            — input-feature correlation
  8.   plots/spatial_error_heatmap.png                  — per-coordinate MAE map of
  9.   plots/feature_distributions.png                  — temperature frequency dis
  10.  plots/residuals_vs_fitted.png                    — residuals vs fitted value
  11.  plots/error_vs_horizon.png                       — MAE/RMSE vs forecast step
  12.  plots/error_by_hour_of_day.png                   — MAE by clock hour of day
"""
import os
import numpy as np
import pandas as pd

import matplotlib
matplotlib.use('Agg')   # headless backend — safe inside Kaggle notebooks
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import r2_score

PLOTS_DIR        = 'plots'
HISTORY_CSV_PATH = 'training_history_temp.csv'

sns.set_theme(style='whitegrid')
_RNG = np.random.default_rng(7)
```

```python
# ── Infrastructure ──────────────────────────────────────────────────────

def ensure_plots_dir(path: str = PLOTS_DIR) -> str:
    os.makedirs(path, exist_ok=True)
    return path


def save_history(history: dict, save_path: str = HISTORY_CSV_PATH) -> None:
    # Keep only the per-epoch list columns (drop scalars like 'best_epoch').
    cols = {k: v for k, v in history.items() if isinstance(v, list)}
    pd.DataFrame(cols).to_csv(save_path, index=False)
    print(f"  Training history -> {save_path}")


def _subsample_flat(*arrays, n: int = 25000):
    """Flatten then take a shared random subset of points (for scatter/hist)."""
    flat = [a.ravel() for a in arrays]
    total = flat[0].size
    if total > n:
        pick = _RNG.choice(total, size=n, replace=False)
        flat = [f[pick] for f in flat]
    return flat


# ── 1-3. Loss curves ────────────────────────────────────────────────────

def plot_loss_curves(history: dict, out_dir: str = PLOTS_DIR) -> None:
    epochs = history['epoch']
    specs = [
        ('mse',  'MSE',  'loss_mse.png'),
        ('rmse', 'RMSE', 'loss_rmse.png'),
        ('mae',  'MAE',  'loss_mae.png'),
    ]
    for key, label, fname in specs:
        fig, ax = plt.subplots(figsize=(10, 5))
        ax.plot(epochs, history[f'train_{key}'], 'o-',  label=f'Train {label}', lin
        ax.plot(epochs, history[f'val_{key}'],   's--', label=f'Val {label}',   lin
        ax.set_xlabel('Epoch')
        ax.set_ylabel(f'{label} (scaled [0, 1] space)')
        ax.set_title(f'LSTM Weekly Temperature — Train vs Val {label}')
        ax.legend()
        ax.set_xticks(epochs)
        fig.tight_layout()
        fig.savefig(os.path.join(out_dir, fname), dpi=150)
        plt.close(fig)
        print(f"  Loss curve       -> {os.path.join(out_dir, fname)}")


# ── 4. Scatter actual vs predicted ──────────────────────────────────────

def plot_scatter_actual_vs_pred(targets_c, preds_c, out_dir: str = PLOTS_DIR) -> No
    a, p = _subsample_flat(targets_c, preds_c)
    r2 = r2_score(targets_c.ravel(), preds_c.ravel())

    lo = min(a.min(), p.min())
```

```
    hi = max(a.max(), p.max())
    pad = (hi - lo) * 0.02
    lo, hi = lo - pad, hi + pad

    fig, ax = plt.subplots(figsize=(7, 7))
    ax.scatter(a, p, s=6, alpha=0.25, edgecolors='none')
    ax.plot([lo, hi], [lo, hi], 'r--', linewidth=2, label='Ideal  y = x')
    ax.set_xlim(lo, hi); ax.set_ylim(lo, hi)
    ax.set_xlabel('Actual temperature (°C)')
    ax.set_ylabel('Predicted temperature (°C)')
    ax.set_title(f'Actual vs Predicted Temperature  (R² = {r2:.3f})')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'scatter_actual_vs_pred.png'), dpi=150)
    plt.close(fig)
    print(f"  Scatter A/P      -> {os.path.join(out_dir, 'scatter_actual_vs_pred.pn


# ── 5. Sample 168h forecast vs actual — South vs North station ──────────────

def _pick_station(coord_ids, coords, lat_lo=None, lat_hi=None, prefer='min'):
    """
    Return (sample_index, coord_id) for a station whose latitude is in
    [lat_lo, lat_hi]. Falls back to the most extreme latitude present when the
    band is empty (prefer='min' → southernmost, 'max' → northernmost).
    """
    present = np.unique(coord_ids)
    lats    = np.array([coords[c][0] for c in present])

    mask = np.ones(len(present), dtype=bool)
    if lat_lo is not None:
        mask &= lats >= lat_lo
    if lat_hi is not None:
        mask &= lats <= lat_hi

    cand = present[mask]
    if len(cand) == 0:                                  # fallback to extreme lat
        cand_lat = lats
        chosen   = present[np.argmin(cand_lat) if prefer == 'min' else np.argmax(ca
    else:
        cand_lat = np.array([coords[c][0] for c in cand])
        chosen   = cand[np.argmin(cand_lat) if prefer == 'min' else np.argmax(cand_

    sample_idx = int(np.where(coord_ids == chosen)[0][0])
    return sample_idx, int(chosen)


def plot_sample_prediction_comparison(targets_c, preds_c, coord_ids, coords,
                                      out_dir: str = PLOTS_DIR) -> None:
    """Two stacked subplots: a stable South station vs a volatile North station."""
    hours = np.arange(targets_c.shape[1])

    # South: lat < 11.5 (weather stable, low error). North: lat > 20.5 (volatile).
    south_i, south_c = _pick_station(coord_ids, coords, lat_hi=11.5, prefer='min')
    north_i, north_c = _pick_station(coord_ids, coords, lat_lo=20.5, prefer='max')
```

```
    panels = [
        (south_i, south_c, 'Miền Nam (ổn định)'),
        (north_i, north_c, 'Miền Bắc/Tây Bắc (biến động)'),
    ]

    fig, axes = plt.subplots(2, 1, figsize=(13, 9), sharex=True)
    for ax, (i, cid, region) in zip(axes, panels):
        lat, lon = coords[cid]
        mae = float(np.mean(np.abs(targets_c[i] - preds_c[i])))
        ax.plot(hours, targets_c[i], color='royalblue', linewidth=2,
                label='Thực tế (actual)')
        ax.plot(hours, preds_c[i], color='deeppink', linestyle='--', linewidth=2,
                label='Dự báo AI (forecast)')
        ax.set_ylabel('Nhiệt độ (°C)')
        ax.set_title(f'{region}  —  (lat={lat:.2f}, lon={lon:.2f})  |  MAE = {mae:.
        ax.legend(loc='upper right')
    axes[-1].set_xlabel('Giờ tương lai (0 → 168h)')
    fig.suptitle('Dự báo nhiệt độ 1 tuần tới — đối chiếu phân hóa địa lý', y=0.995)
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'sample_prediction_comparison.png'), dpi=150)
    plt.close(fig)
    print(f"  Sample forecast  -> {os.path.join(out_dir, 'sample_prediction_compari


# ── 6. Residuals histogram ──────────────────────────────────────────

def plot_residuals_histogram(targets_c, preds_c, out_dir: str = PLOTS_DIR) -> None:
    resid = (targets_c - preds_c).ravel()
    (resid,) = _subsample_flat(resid, n=60000)

    fig, ax = plt.subplots(figsize=(9, 5))
    sns.histplot(resid, bins=80, kde=True, ax=ax, color='steelblue')
    ax.axvline(0, color='red', linestyle='--', linewidth=2, label='Zero error')
    ax.set_xlabel('Residual = Actual − Predicted (°C)')
    ax.set_ylabel('Frequency')
    ax.set_title(f'Residual Distribution  (mean = {resid.mean():.4f} °C)')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'residuals_histogram.png'), dpi=150)
    plt.close(fig)
    print(f"  Residual hist    -> {os.path.join(out_dir, 'residuals_histogram.png')


# ── 7. Feature correlation heatmap ──────────────────────────────────

def plot_feature_correlation_heatmap(df_sample, feature_cols,
                                     target_col: str = 'temperature_celsius',
                                     out_dir: str = PLOTS_DIR) -> None:
    cols = [c for c in feature_cols if c in df_sample.columns]
    corr = df_sample[cols].corr()

    fig, ax = plt.subplots(figsize=(14, 12))
    sns.heatmap(corr, cmap='coolwarm', center=0, square=True,
                linewidths=0.4, cbar_kws={'shrink': 0.8}, ax=ax)
    ax.set_title(f'Input Feature Correlation Matrix (target: {target_col})')
    fig.tight_layout()
```

```
    fig.savefig(os.path.join(out_dir, 'feature_correlation_heatmap.png'), dpi=150)
    plt.close(fig)
    print(f"  Corr heatmap     -> {os.path.join(out_dir, 'feature_correlation_heatm

# ── 8. Spatial error heatmap ──────────────────────────────────────────

def plot_spatial_error_heatmap(targets_c, preds_c, coord_ids, coords,
                               out_dir: str = PLOTS_DIR) -> None:
    abs_err = np.abs(targets_c - preds_c).mean(axis=1)   # per-sample MAE

    lats, lons, maes = [], [], []
    for cid in np.unique(coord_ids):
        mask = coord_ids == cid
        lat, lon = coords[cid]
        lats.append(lat); lons.append(lon)
        maes.append(float(abs_err[mask].mean()))

    fig, ax = plt.subplots(figsize=(8, 10))
    sc = ax.scatter(lons, lats, c=maes, cmap='YlOrRd', s=60,
                    edgecolors='k', linewidths=0.3)
    fig.colorbar(sc, ax=ax, label='Mean Absolute Error (°C)')
    ax.set_xlabel('Longitude'); ax.set_ylabel('Latitude')
    ax.set_title('Spatial Temperature Forecast Error Across Vietnam')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'spatial_error_heatmap.png'), dpi=150)
    plt.close(fig)
    print(f"  Spatial error    -> {os.path.join(out_dir, 'spatial_error_heatmap.png

# ── 9. Temperature distribution (natural bell shape) ──────────────────

def plot_feature_distributions(raw_temp, out_dir: str = PLOTS_DIR) -> None:
    temp = np.asarray(raw_temp, dtype=np.float64)
    temp = temp[np.isfinite(temp)]

    fig, ax = plt.subplots(figsize=(10, 6))
    sns.histplot(temp, bins=80, kde=True, ax=ax, color='darkorange')
    ax.axvline(temp.mean(), color='red', linestyle='--', linewidth=2,
               label=f'Mean = {temp.mean():.2f} °C')
    ax.set_xlabel('Temperature (°C)')
    ax.set_ylabel('Frequency')
    ax.set_title('Distribution of Actual Temperature '
                 '(naturally bell-shaped — no log transform needed)')
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'feature_distributions.png'), dpi=150)
    plt.close(fig)
    print(f"  Distributions    -> {os.path.join(out_dir, 'feature_distributions.png

# ── 10. Residuals vs fitted ───────────────────────────────────────────

def plot_residuals_vs_fitted(targets_c, preds_c, out_dir: str = PLOTS_DIR) -> None:
    fitted, resid = _subsample_flat(preds_c, targets_c - preds_c)
```

```
    fig, ax = plt.subplots(figsize=(9, 6))
    ax.scatter(fitted, resid, s=6, alpha=0.25, edgecolors='none')
    ax.axhline(0, color='red', linestyle='--', linewidth=2)
    ax.set_xlabel('Fitted / predicted temperature (°C)')
    ax.set_ylabel('Residual = Actual − Predicted (°C)')
    ax.set_title('Residuals vs Fitted Values')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'residuals_vs_fitted.png'), dpi=150)
    plt.close(fig)
    print(f"  Resid vs fitted  -> {os.path.join(out_dir, 'residuals_vs_fitted.png')


# ── 11. Error vs forecast horizon ──────────────────────────────────────

def plot_error_vs_horizon(targets_c, preds_c, out_dir: str = PLOTS_DIR) -> None:
    """How accuracy decays as we forecast further into the week."""
    diff   = targets_c - preds_c                    # (M, 168)
    mae_h  = np.mean(np.abs(diff), axis=0)          # (168,)
    rmse_h = np.sqrt(np.mean(diff ** 2, axis=0))    # (168,)
    steps  = np.arange(1, targets_c.shape[1] + 1)

    fig, ax = plt.subplots(figsize=(12, 5))
    ax.plot(steps, mae_h,  color='steelblue',  linewidth=2, label='MAE  (°C)')
    ax.plot(steps, rmse_h, color='indianred',  linewidth=2, label='RMSE (°C)')
    # day boundaries help read the diurnal structure of the error
    for d in range(24, targets_c.shape[1], 24):
        ax.axvline(d, color='grey', linestyle=':', linewidth=0.6, alpha=0.6)
    ax.set_xlabel('Bước dự báo tương lai (giờ, 1 → 168)')
    ax.set_ylabel('Sai số trung bình (°C)')
    ax.set_title('Sai số theo tầm dự báo — độ chính xác suy giảm khi dự báo càng xa
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'error_vs_horizon.png'), dpi=150)
    plt.close(fig)
    print(f"  Error vs horizon -> {os.path.join(out_dir, 'error_vs_horizon.png')}")


# ── 12. Error by hour of day ──────────────────────────────────────

def plot_error_by_hour_of_day(targets_c, preds_c, hours_of_day,
                              out_dir: str = PLOTS_DIR) -> None:
    """Which clock hours the model forecasts worst (e.g. midday heat, late night)."
    abs_err = np.abs(targets_c - preds_c).ravel()
    hours   = np.asarray(hours_of_day).ravel()

    mae_by_hour = np.array([
        abs_err[hours == h].mean() if np.any(hours == h) else np.nan
        for h in range(24)
    ])

    fig, ax = plt.subplots(figsize=(11, 5))
    bars = ax.bar(np.arange(24), mae_by_hour, color='darkorange',
                  edgecolor='black', linewidth=0.4)
    worst = int(np.nanargmax(mae_by_hour))
    bars[worst].set_color('crimson')
    ax.set_xticks(np.arange(24))
```

```
    ax.set_xlabel('Khung giờ trong ngày (0 → 23)')
    ax.set_ylabel('MAE trung bình (°C)')
    ax.set_title(f'Sai số theo giờ trong ngày — kém nhất lúc {worst:02d}:00 '
                 f'({mae_by_hour[worst]:.2f} °C)')
    fig.tight_layout()
    fig.savefig(os.path.join(out_dir, 'error_by_hour_of_day.png'), dpi=150)
    plt.close(fig)
    print(f"  Error by hour    -> {os.path.join(out_dir, 'error_by_hour_of_day.png'


# ── Orchestrator ──────────────────────────────────────────────────────

def save_all_plots(history, targets_c, preds_c, coord_ids, coords,
                   df_sample, feature_cols, raw_temp, hours_of_day,
                   target_col: str = 'temperature_celsius',
                   out_dir: str = PLOTS_DIR) -> None:
    ensure_plots_dir(out_dir)
    plot_loss_curves(history, out_dir)
    plot_scatter_actual_vs_pred(targets_c, preds_c, out_dir)
    plot_sample_prediction_comparison(targets_c, preds_c, coord_ids, coords, out_di
    plot_residuals_histogram(targets_c, preds_c, out_dir)
    plot_feature_correlation_heatmap(df_sample, feature_cols, target_col, out_dir)
    plot_spatial_error_heatmap(targets_c, preds_c, coord_ids, coords, out_dir)
    plot_feature_distributions(raw_temp, out_dir)
    plot_residuals_vs_fitted(targets_c, preds_c, out_dir)
    plot_error_vs_horizon(targets_c, preds_c, out_dir)
    plot_error_by_hour_of_day(targets_c, preds_c, hours_of_day, out_dir)
```

In [ ]:

```
%%writefile /kaggle/working/lstm_temp_weekly/main.py
"""
Task: Orchestrate the full weekly-TEMPERATURE pipeline:
      Load → Engineer → Scale → Sequences → Train → Evaluate → Plots → Checkpoint.

Model: Seq2Seq Encoder-Decoder LSTM (vectorized teacher forcing in training,
       autoregressive at inference), with AMP, early stopping, and a checkpoint
       saved every epoch to models/lstm_temp_weekly_epoch_NN.pt.

Run on Kaggle (GPU):
    !python /kaggle/working/lstm_temp_weekly/main.py
"""
import os
import sys

# Allow sibling-package imports regardless of the current working directory.
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

from data.load_data import load_data
from preprocessing.feature_engineering import engineer_features, TARGET_COL
from preprocessing.scaling import fit_and_scale, SCALER_PATH, FEATURE_COLS_PATH
from dataset.sequence_dataset import (
    build_coordinate_arrays,
```

```
    build_split_indices,
    resolve_decoder_feat_idx,
    WeatherSequenceDataset,
    DECODER_FEATURE_COLS,
    SEQUENCE_LENGTH,
    PREDICT_STEPS,
)
from model.lstm_model import LSTMModel
from training.trainer import train_model
from training.evaluator import evaluate_test, collect_predictions
from visualization.plot_loss import save_history, save_all_plots

# ── Hyper-parameters ──────────────────────────────────────────────────────
EPOCHS      = 5         # max epochs; early stopping usually halts sooner
PATIENCE    = 2         # stop when val MSE doesn't improve for 2 epochs
BATCH_SIZE  = 2048      # large batch to saturate the Kaggle T4 GPU
HIDDEN_SIZE = 128
NUM_LAYERS  = 2
DROPOUT     = 0.1
LR          = 0.001
USE_AMP     = True      # mixed precision on CUDA

MODELS_DIR  = 'models'
CKPT_PREFIX = 'lstm_temp_weekly'
MODEL_PATH  = os.path.join(MODELS_DIR, 'lstm_temp_weekly_model.pt')


def main() -> None:
    # ── Device ────────────────────────────────────────────────────────────
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f"Device : {device}")
    if device.type == 'cuda':
        print(f"GPU    : {torch.cuda.get_device_name(0)}")
    use_pin = device.type == 'cuda'

    # ── 1. Load ───────────────────────────────────────────────────────────
    print("\n[1/7] Loading data ...")
    df = load_data()

    # Keep a raw-temperature sample for the distribution plot (it stays in °C
    # the whole pipeline, but we grab it before scaling for a clean axis).
    raw_temp = df[TARGET_COL].sample(
        n=min(100_000, len(df)), random_state=0
    ).to_numpy()

    # ── 2. Feature engineering ────────────────────────────────────────────
    print("\n[2/7] Engineering features ...")
    df = engineer_features(df)
    print(f"  {len(df):>12,} rows  |  {len(df.columns)} columns")

    # ── 3. Scale ──────────────────────────────────────────────────────────
    print("\n[3/7] Scaling ...")
    df, scaler, feature_cols = fit_and_scale(df)
    target_idx       = feature_cols.index(TARGET_COL)
    decoder_feat_idx = resolve_decoder_feat_idx(feature_cols)
    dec_feat_size    = len(decoder_feat_idx)
```

```python
# Sample of the scaled frame for the correlation heatmap (before we drop df).
df_sample = df[feature_cols].sample(
    n=min(50_000, len(df)), random_state=0
).copy()

# ── 4. Sequences ──────────────────────────────────────────────────────
print("\n[4/7] Building sequences ...")
coordinate_arrays, coords = build_coordinate_arrays(df, feature_cols)
del df   # release the big frame; arrays + sample are all we still need

train_idx, test_idx = build_split_indices(
    coordinate_arrays, SEQUENCE_LENGTH, PREDICT_STEPS
)
print(f"  Train: {len(train_idx):,}  |  Test: {len(test_idx):,}  "
      f"(seq_len={SEQUENCE_LENGTH}, predict_steps={PREDICT_STEPS}, "
      f"dec_feat={dec_feat_size})")

train_ds = WeatherSequenceDataset(
    coordinate_arrays, train_idx, target_idx, decoder_feat_idx,
    SEQUENCE_LENGTH, PREDICT_STEPS,
)
test_ds = WeatherSequenceDataset(
    coordinate_arrays, test_idx, target_idx, decoder_feat_idx,
    SEQUENCE_LENGTH, PREDICT_STEPS,
)

train_loader = DataLoader(
    train_ds, batch_size=BATCH_SIZE, shuffle=True,
    num_workers=2, pin_memory=use_pin, persistent_workers=True,
)
# The test split doubles as the per-epoch validation monitor.
val_loader = DataLoader(
    test_ds, batch_size=BATCH_SIZE, shuffle=False,
    num_workers=2, pin_memory=use_pin, persistent_workers=True,
)

# ── 5. Model ──────────────────────────────────────────────────────────
input_size = len(feature_cols)
model = LSTMModel(
    input_size, dec_feat_size, HIDDEN_SIZE, NUM_LAYERS,
    PREDICT_STEPS, DROPOUT, target_idx,
).to(device)
optimizer = optim.Adam(model.parameters(), lr=LR)
criterion = nn.MSELoss()

# Full network config — enough to rebuild the model from any checkpoint.
config = {
    'input_size':        input_size,
    'dec_feat_size':     dec_feat_size,
    'hidden_size':       HIDDEN_SIZE,
    'num_layers':        NUM_LAYERS,
    'predict_steps':     PREDICT_STEPS,        # 168
    'output_size':       PREDICT_STEPS,        # alias (back-compat)
    'dropout':           DROPOUT,
    'target_idx':        target_idx,
```

```
    'sequence_length':   SEQUENCE_LENGTH,      # 168
    'decoder_feature_cols': DECODER_FEATURE_COLS,
    'target_cols':       [TARGET_COL],         # ['temperature_celsius']
    'scaler_name':       SCALER_PATH,          # 'scaler_temp.pkl'
    'feature_cols_name': FEATURE_COLS_PATH,    # 'feature_cols_temp.pkl'
}

n_params = sum(p.numel() for p in model.parameters())
print(f"\n[5/7] Seq2Seq model ready  |  input_size={input_size}  |  "
      f"dec_feat_size={dec_feat_size}  |  predict_steps={PREDICT_STEPS}  |  "
      f"params={n_params:,}")

# ── 6. Train ──────────────────────────────────────────────────────────
print(f"\n[6/7] Training  (max {EPOCHS} epochs, patience={PATIENCE}, "
      f"batch={BATCH_SIZE}) ...")
print("-" * 100)
history = train_model(
    model, train_loader, val_loader, optimizer, criterion, device,
    epochs=EPOCHS, patience=PATIENCE, config=config,
    models_dir=MODELS_DIR, ckpt_prefix=CKPT_PREFIX, use_amp=USE_AMP,
)
print("-" * 100)

# ── 7. Evaluate (real units) + visualise + export ─────────────────────
print("\n[7/7] Evaluating on the test set (autoregressive) ...")
evaluate_test(model, test_ds, device, scaler, target_idx, BATCH_SIZE)

print("\n[Export]")
save_history(history)

preds_c, targets_c, coord_ids, hours_of_day = collect_predictions(
    model, coordinate_arrays, test_idx, target_idx, decoder_feat_idx,
    feature_cols, scaler, SEQUENCE_LENGTH, PREDICT_STEPS, device,
    max_samples=4000, batch_size=BATCH_SIZE,
)
save_all_plots(
    history, targets_c, preds_c, coord_ids, coords,
    df_sample, feature_cols, raw_temp, hours_of_day, target_col=TARGET_COL,
)

# ── Final checkpoint (best epoch's weights) ───────────────────────────
os.makedirs(MODELS_DIR, exist_ok=True)
torch.save(
    {
        'model_state_dict': model.state_dict(),
        'config':           config,
        'best_epoch':       history.get('best_epoch'),
        # back-compat top-level keys
        'input_size':        input_size,
        'hidden_size':       HIDDEN_SIZE,
        'num_layers':        NUM_LAYERS,
        'output_size':       PREDICT_STEPS,
        'dropout':           DROPOUT,
        'sequence_length':   SEQUENCE_LENGTH,
        'target_cols':       [TARGET_COL],
        'scaler_name':       SCALER_PATH,
```

```
            'feature_cols_name': FEATURE_COLS_PATH,
        },
        MODEL_PATH,
    )
    print(f"  Checkpoint       -> {MODEL_PATH}")
    print("\n[Done]")


if __name__ == '__main__':
    main()
```

Huấn luyện

Chạy toàn bộ pipeline: Load → Engineer → Scale → Sequences → Train → Evaluate → 8 Plots → Checkpoint.

In [ ]:
```
!python /kaggle/working/lstm_temp_weekly/main.py
```

Xem các biểu đồ kết quả

In [ ]:
```
from IPython.display import Image, display
import os

plot_files = [
    'loss_mse.png', 'loss_rmse.png', 'loss_mae.png',
    'scatter_actual_vs_pred.png', 'sample_prediction_comparison.png',
    'residuals_histogram.png', 'feature_correlation_heatmap.png',
    'spatial_error_heatmap.png', 'feature_distributions.png',
    'residuals_vs_fitted.png',
    'error_vs_horizon.png', 'error_by_hour_of_day.png',
]
for name in plot_files:
    path = os.path.join('plots', name)
    if os.path.exists(path):
        print(name)
        display(Image(filename=path))
```